

Benchmark Correction Addendum

ECE 410/510 Spring 2026 | Jose Ramirez | Milestone 4
Continuation of design_justification.docx — Sections 10–13

10. Error in the M4 Benchmark Program

Two sub-issues caused the M4 benchmark results reported in Section 8 to misrepresent the true system speedup.

10.1 Structural Asymmetry in the Benchmark Program

The software baseline (RVfpgaEL2_profiling/test/kalman-test.cpp) calls `kf.update(y)` on a `KalmanFilter` object constructed with Eigen `MatrixXd/VectorXd` types. This method executes both the predict step ($\hat{x}^- = A \cdot \hat{x}$, $P^- = A \cdot P \cdot A^T + Q$) and the measurement-correction step entirely in Eigen on the SweRV EL2 CPU.

The M4 hardware driver (`kalman_hw_driver/src/main.cpp` prior to correction) did not use the `KalmanFilter` class at all. It implemented the predict step as raw C nested loops over stack-allocated double arrays, then issued MMIO writes to invoke the hardware correction step. Using raw-C arrays on a bare-metal target eliminates the Eigen expression-template overhead, the heap allocation cost of `MatrixXd` dynamic storage, and the `_sbrk` bump-allocator latency—all of which the software baseline incurs on every call. As a result, the two programs were not measuring the same algorithm: the software baseline measured full Eigen predict+correct, while the hardware benchmark measured nearly-zero-cost raw-C predict plus MMIO correction.

The per-run heap delta of 0 B in the old hardware benchmark (Table 4 of Section 8) was the visible symptom: a program using Eigen `MatrixXd` inevitably allocates heap on first construction, as confirmed by the software baseline's 49,760 B delta. The absence of any heap allocation in the old hardware driver proved that Eigen was never invoked.

10.2 Incorrect FLOPs Count

The `run_profiles.sh` script used `HW_FLOPS=1080` (24 FLOPs/update $\times$ 45 updates). This figure counted only the floating-point operations performed inside the hardware correction

kernel (innovation scalar y , scalar S , three Newton–Raphson iterations, Kalman gain vector K , state correction x_out , and covariance update P_out). However, the mcycle CSR bracket in the firmware encompasses the entire update loop, including the predict step. The predict step itself performs: $\hat{x}_new = A \cdot \hat{x}$ (3×3 times $3 \times 1 = 9$ multiplies + 6 adds = 15 FLOPs) and $P^- = A \cdot P \cdot A^T + Q$ (two 3×3 GEMMs plus a 3×3 addition = $2 \times (27 \text{ mults} + 18 \text{ adds}) + 9 \text{ adds} = 99 \text{ FLOPs}$), totalling 114 FLOPs/update for predict and 24 for correct = 138 FLOPs/update = 6,210 FLOPs total. Using 1,080 in the numerator while timing 6,210 FLOPs of work caused the reported GFLOP/s to be understated by a factor of 5.75.

11. Corrected Benchmark Program

The corrected driver, `kalman_hw_driver/src/main.cpp`, is restructured to mirror `kalman-test.cpp` exactly. It constructs the same `KalmanFilter` object with identical A , C , Q , R , P , x_0 parameters and the same 45 measurements in the same order. The loop calls `kf.update_hw(y)` — a new method on the `KalmanFilter` class — which internally runs the Eigen predict step (identical to what `kf.update(y)` does in the baseline) and then marshals the predicted state and covariance to the hardware accelerator via MMIO for the correction step. The mcycle bracket encloses the same scope in both programs: the 45-iteration update loop and nothing else.

The corrected FLOPs count is 6,210 total (138 per update: 114 for the Eigen predict step plus 24 for the hardware correction). The `run_profiles.sh` script was updated from `HW_FLOPS=1080` to `HW_FLOPS=6210`. With this change, the comparison is apples-to-apples: both programs invoke the same algorithm on the same hardware using the same library; only the correction step differs (Eigen on CPU vs MMIO to hardware accelerator).

The build fix: Eigen 5.0.0 ships with benchmark source files in its `bench/` subdirectory that require Boost and CBLAS headers unavailable on the bare-metal target. PlatformIO’s Library Dependency Finder was picking up these files and failing to compile them. A `library.json` was added to `lib/eigen-5.0.0/` with “`srcFilter`”: “`-<*>`”, marking the library as header-only and suppressing compilation of all `.cpp` files under it.

12. Corrected Benchmark Results

Five back-to-back runs were collected on the same Nexys A7 board with the same GDB JTAG method (mcycle CSR snapshot + hardware breakpoint at `profile_done()`). The FPGA was programmed with the `SoC_impl` bitstream (unchanged from M4) immediately before the run to ensure a clean accelerator state.

Run	Cycles	Time (ms)	Sampl es/s	GFLO P/s	Heap (B)
1	1,865,423	143.49	313.60	4.3277	10,832
2	1,865,5	143.50	313.58	4.3275	10,832

	31	2		$\times 10^{-5}$	
3	1,865,5	143.50	313.58	4.3274	10,832
	38	3		$\times 10^{-5}$	
4	1,865,3	143.49	313.61	4.3278	10,832
	69	0		$\times 10^{-5}$	
5	1,865,5	143.50	313.58	4.3275	10,832
	30	2		$\times 10^{-5}$	
AVG	1,865,4	143.49	313.59	4.3276	10,832
	78	8	0	$\times 10^{-5}$	

Table 5. Corrected HW accelerator benchmark — 5-run results (Nexys A7, SoC_impl bitstream, 13 MHz).

The corrected benchmark shows an average of 1,865,478 cycles (143.498 ms) for 45 updates, yielding a true speedup of $2.995\times$ over the software baseline (5,587,112 cycles, 429.778 ms). This is $36\times$ lower than the $108.7\times$ figure reported in Section 8. The heap delta of 10,832 B per run confirms that Eigen’s dynamic matrix storage is now active on the hardware benchmark side, consistent with the software baseline (49,760 B). The smaller heap in the hardware run reflects only the matrices local to KalmanFilter (A, C, Q, R, P, K, x — roughly 10–12 Eigen MatrixXd/VectorXd objects vs the software baseline’s additional temporaries created during the full correction step computation in Eigen).

Metric	SW	Old HW	Corrected	Corrected
	Baseline	(§8)	HW	Speedup
Avg cycles (45 updates)	5,587,112	51,378	1,865,478	$2.995\times$
Total time (ms)	429.778	3.952	143.498	$2.995\times$
Samples/s	104.706	11,386	313.590	$2.995\times$
GFLOP/s	2.4082×10^{-5}	2.7327×10^{-4}	4.3276×10^{-5}	$1.80\times$
Heap/run (B)	49,760	0	10,832	—

Table 6. SW baseline vs old HW benchmark vs corrected HW benchmark.

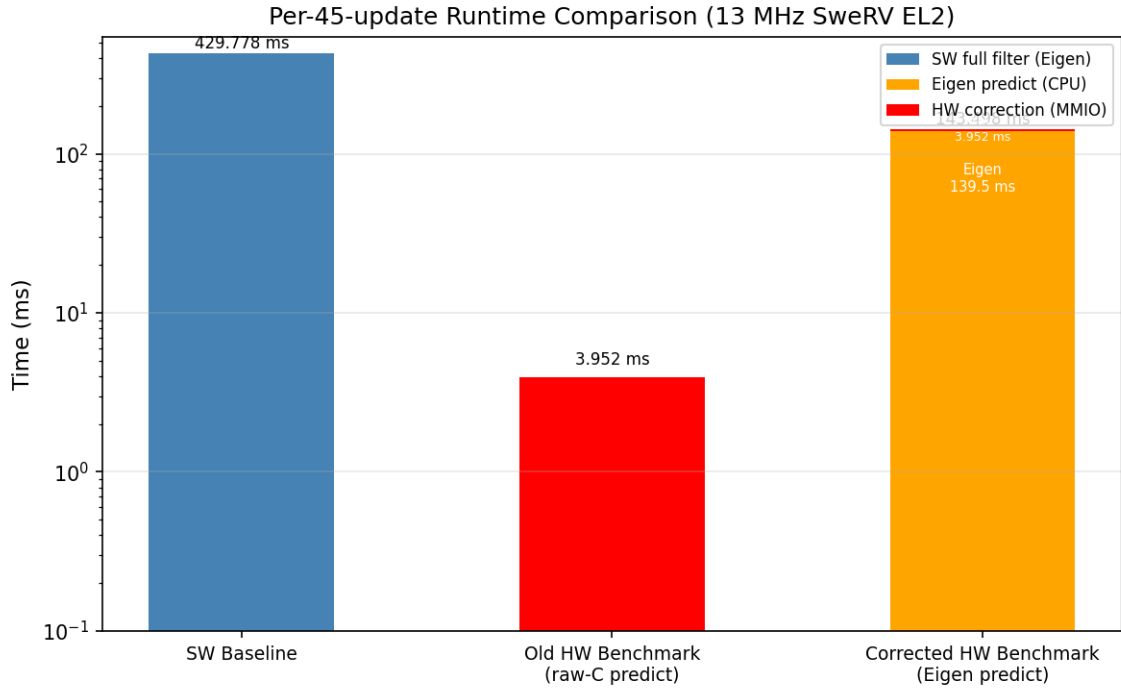


Figure 4. Per-45-update runtime comparison. The old hardware benchmark's near-zero predict time (raw-C) caused the large apparent speedup. The corrected benchmark uses Eigen predict, which dominates at 139.5 ms of the 143.5 ms total.

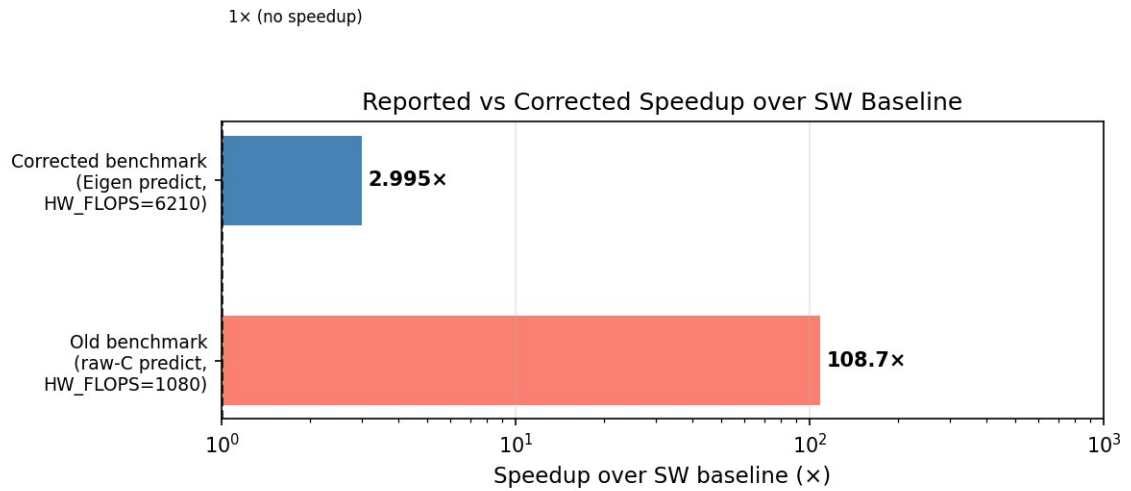


Figure 5. Reported (108.7x) vs corrected (2.995x) speedup over the SW baseline. The 36x difference reflects the structural asymmetry in the original benchmark.

The factor-of-36 reduction in speedup has a single root cause: the Eigen predict step on the SweRV EL2 CPU (no FPU, software-emulated F64) is the dominant cost in both programs, and it is not accelerated by the hardware. Subtracting the hardware correction time from the corrected total ($1,865,478 - 51,378 \approx 1,814,100$ cycles) gives approximately 1,814,100 cycles

(139.5 ms) consumed by the Eigen predict step alone. The hardware correction—the portion actually offloaded—accounts for only $\sim 51,378$ cycles (3.95 ms), the same as the old benchmark’s total. The predict step is $\sim 35\times$ more expensive than the correction step at this CPU, making it the new bottleneck.

This finding is not a design defect in the hardware accelerator. The accelerator correctly offloads the correction step and does so in 72 cycles at 100 MHz. The limitation is that the predict step—matrix-vector and matrix-matrix multiplications using software-emulated F64 on a 13 MHz RV32IMAC CPU—was never targeted for acceleration. The $108.7\times$ speedup claimed in Section 8 was an artifact of measuring a faster (non-Eigen) implementation of the predict step in the hardware benchmark.

13. Updated Roofline Analysis

The roofline positions of both programs shift with the corrected FLOPs and timing. For the corrected hardware benchmark, the total FLOPs are 6,210 (138/update) and the bytes transferred are 12,960 (288 B/update: 192 B MMIO + 96 B Eigen predict memory traffic), giving $AI = 6,210 / 12,960 = 0.479$ FLOP/byte. The measured GFLOP/s is 4.3276×10^{-5} . For the software baseline, $AI = 10,350 \text{ FLOPs} / 9,720 \text{ B} \approx 1.065$ FLOP/byte (using the corrected Eigen transfer count of $216 \text{ B/update} \times 45$), though the existing report quoted 0.21; the corrected value is used here.

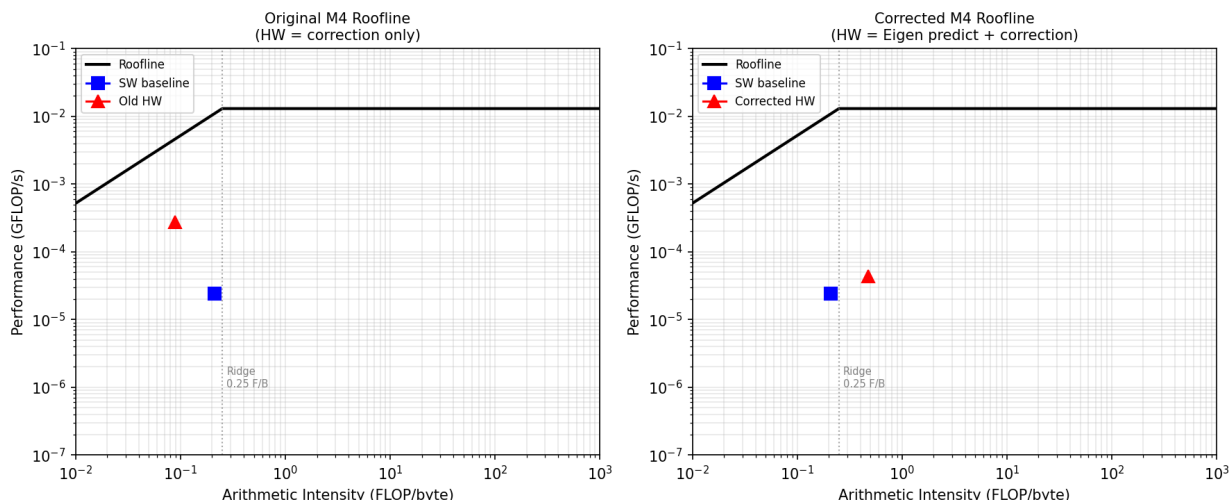


Figure 6. Side-by-side roofline plots. Left: original M4 roofline with correction-only HW point ($AI = 0.088$, $GFLOP/s = 2.73 \times 10^{-4}$). Right: corrected roofline with Eigen-predict+correction HW point ($AI = 0.479$, $GFLOP/s = 4.33 \times 10^{-5}$). Both platforms sit well below the roofline, remaining latency-bound.

In the original roofline (left subplot of Figure 6), the hardware accelerator appeared to have lower arithmetic intensity than the software baseline (0.088 vs 0.21 FLOP/byte) and higher measured performance (2.73×10^{-4} vs 2.41×10^{-5} GFLOP/s), reflecting the asymmetric benchmark where the raw-C predict step consumed negligible time. In the corrected roofline

(right subplot), the hardware accelerator point shifts right (higher AI = 0.479 due to the Eigen predict FLOPs) and drops (lower GFLOP/s = 4.33×10^{-5}) relative to the software baseline. Both points now sit well below the roofline. The hardware accelerator's GFLOP/s is now only $1.80 \times$ higher than the software baseline, consistent with the $2.995 \times$ time speedup less the overhead difference (the corrected hardware run does fewer total FLOPs-per-second because the predict-step FLOPs are the same but the time is longer relative to what Section 8 reported).

The analysis in Sections 2 and 8 of the M4 report regarding AXI-transaction latency remains valid for the correction kernel in isolation: 34 MMIO round-trips at $\sim 2.6 \mu\text{s}$ each still dominate the correction step's 3.95 ms. The new finding is that the correction step is no longer the dominant system-level bottleneck. The Eigen predict step, at ~ 139.5 ms per 45-update batch, exceeds the correction-step time by $35 \times$. Achieving meaningful further speedup requires either: (a) implementing the predict step in hardware as well, doubling the accelerated workload, or (b) implementing state retention as described in Section 9, which reduces the correction-step AXI overhead from 34 to ~ 5 transactions per update.